

Rikka-mene — Manuale di progetto

Agente AI on-device per Vivo X300, basato su RikkaHub, con AICore / Gemini Nano come motore principale.

Aggiornato al 26 settembre 2026 · branch `claude/fastpathrouter-storage-test-9pwqjf` · ultima APK `apk-b9e1544-run6`

1. In una pagina

Obiettivo. Un'app Android che, sul Vivo X300, porti a termine compiti in più passi (cerca, leggi, estrai, scrivi, verifica) usando il modello locale Gemini Nano tramite AICore, senza cloud.

Il problema centrale. AICore accetta al massimo circa 4.000 token in ingresso e 256 in uscita per richiesta. Una pagina web, un file o pochi passi di un'agente superano questo limite subito.

La soluzione. Non si allarga il limite (non è possibile): il runtime decide cosa entra nel prompt. Tiene sempre il compito dell'utente, dà priorità ai passi più recenti, taglia i risultati lunghi tenendo le parti pertinenti alla domanda, conserva i dati completi fuori dal prompt e permette al modello di recuperarli con un tool dedicato.

Stato. APK debug installabile (si chiama **Rikka-mene**), aggiornamenti in-place, build automatica su GitHub. Sul Vivo, AICore ha completato un compito reale con strumenti (lettura di una pagina web e risposta corretta). I modelli locali alternativi (LiteRT / llama.cpp) funzionano ma sono risultati troppo lenti ed energivori: AICore resta il motore dell'app.

Cosa	Stato
Build APK debug	Verificato (sandbox cloud e GitHub Actions)
Installazione e avvio sul Vivo	Verificato sul Vivo
Chat AICore / Gemini Nano FULL	Verificato sul Vivo
Ciclo agentico con tool reali (web_fetch)	Verificato sul Vivo
Gestione del contesto (task lungo)	Verificato con test automatici; sul Vivo su 1 task
Task multi-step (3+ tool), recupero errori, loop	Da verificare sul Vivo (prompt nel capitolo 7)
Modelli locali LiteRT / llama.cpp	Funzionanti ma poco usabili (lenti, consumano)

2. Come funziona (architettura essenziale)

Il flusso di ogni passo dell'agente:

- 1 **Utente** scrive il compito nella chat.
- 2 **GenerationLoop** (il ciclo dell'agente) prepara i tool disponibili e chiede una risposta al modello.
- 3 **AICoreProvider** costruisce il prompt dentro il budget: istruzioni + elenco tool + cronologia compattata. Misura i token con `countTokens` prima di inviare.
- 4 **Gemini Nano** risponde con testo oppure con una chiamata tool `<tool_call>{...}</tool_call>`.
- 5 **Parser** riconosce la chiamata (anche se spezzata in più pezzi di streaming o dal limite di 256 token).
- 6 **Sicurezza**: HardlineCommandGuard, PathSafetyGuard, protezione SSRF, approvazione dell'utente dove richiesta.
- 7 **Tool** esegue davvero l'azione; il risultato completo viene salvato.
- 8 **Gestore del contesto** decide come il risultato entra nel prompt del passo successivo (intero, tagliato con riferimenti, o riassunto nel registro).
- 9 Si ripete fino alla risposta finale, con limiti di passi, di tempo e protezione dai loop.

Principio: il modello propone, il runtime decide ed esegue. Il modello non ha mai accesso diretto a shell o file senza passare dalle regole di sicurezza.

3. Cronologia del lavoro

Fase	Cosa	Esito
Sprint 1-4	Budget del prompt AICore, continuazione oltre i 256 token, sicurezza (symlink, segreti app, rete locale, approvazioni tra contesti, sub-agent)	Base di partenza
Diagnosi test	1 test fallito su 1732: formattazione dipendente dalla lingua del PC	Corretto (Locale.R00T)
Sprint 5	Registro dei passi, read_tool_output , archivio privato dei risultati grandi	Loop eliminato nei test (vedi 4.2)
Sprint 5b	Prima build Android nel cloud; budget esatto con countTokens / getTokenLimit	APK prodotta; firme API verificate sull'AAR reale
Pubblicazione	Workflow GitHub che compila e pubblica l'APK; chiave debug fissa; nome Rikka-mene	Aggiornamenti in-place
Vivo run 1	Test Mura di Lucca: Nano sceglie raw , la pagina non arriva	Corretto: valori degli argomenti visibili al modello
Sprint 6	Gestore del contesto unico anche per LiteRT e llama.cpp; tetto 32K	Bug di crash eliminato
Vivo run 2	Nano legge la pagina ma il nome è nella parte tagliata	Corretto: taglio mirato alla domanda
Vivo run 3	Nano completa il compito	Successo

4. Problemi incontrati e soluzioni

4.1 Test che falliva solo sul PC italiano

Problema. **FastPathRouterTest** falliva sul Dell: atteso "16.0 GB", ottenuto "16,0 GB". **Causa.** La formattazione dei numeri seguiva la lingua del sistema (italiano usa la virgola), ma la frase è in inglese. **Soluzione.** Formato con **Locale.R00T**. Il test ora passa sia in italiano sia in inglese.

4.2 Il modello "dimenticava" cosa aveva già fatto

Problema. Quando la cronologia non entrava nei 4K token, i passi vecchi venivano sostituiti da una sola riga "[earlier steps omitted]". Il modello non sapeva più cosa aveva già fatto. **Evidenza.** In uno scenario automatico da 45 passi, con il vecchio codice il modello rileggeva le stesse cartelle 5-6 volte e si fermava al limite di 80 passi senza risposta. **Soluzione.** Un **registro dei passi**: ogni passo scartato lascia una riga compatta (tool, argomenti, esito). Chiamate ripetute dello stesso tool diventano una riga sola (es. **list_dir x17: dir_1 ... dir_17 -> 17 ok**). Anche le note scritte dal modello durante il compito vengono conservate. **Risultato.** Stesso scenario: 46 richieste, nessuna ripetizione, trovato il dato scoperto 28 passi prima.

4.3 Parti tagliate dei risultati irrecuperabili

Problema. Un risultato lungo veniva tagliato nel mezzo e quella parte era persa per sempre. **Soluzione.** Nuovo tool di sistema **read_tool_output(id, offset, query)**: legge a pagine o cerca righe nel risultato completo di una chiamata precedente. Il segno di taglio dice esattamente dove riprendere (**read_tool_output id=... offset=...**). **Sicurezza.** Solo lettura, nessuna approvazione; legge solo risultati della **stessa conversazione** (id di altre chat rifiutati); i risultati molto grandi (oltre 32 KB) sono salvati in un archivio privato non accessibile alla shell né ai tool sui file, e cancellato a ogni avvio dell'app.

4.4 Budget dei token misurato, non stimato

Problema. Il prompt era dimensionato con una stima (circa 3 caratteri per token). Una stima sbagliata porta a richieste rifiutate o a spazio sprecato. **Soluzione.** Usate le API reali di ML Kit (verificate sul file AAR): `countTokens` misura ogni richiesta prima dell'invio e il prompt viene ricostruito se serve; `getTokenLimit` può solo abbassare il limite documentato. Se le API falliscono resta la stima con un tentativo di ripiego. **Risultato nei test.** Con un conteggio reale 1,5 volte più alto della stima, nessuna richiesta oltre il limite; con uno più basso, il prompt tiene 23 risultati invece di 12.

4.5 Nano non sapeva quali valori usare negli argomenti (Vivo run 1)

Problema. Nano chiamava `web_fetch` con `extract_mode: "raw"` (HTML grezzo, massimo 8 KB): su Wikipedia sono solo intestazioni tecniche, il testo della pagina non arrivava mai. **Causa.** Nel prompt AI Core ogni tool mostrava solo i nomi degli argomenti, non i valori ammessi. **Soluzione.** Il prompt mostra i valori ammessi (`extract_mode=article|raw|text|links|metadata`); `web_fetch` avvisa nel risultato quando l'HTML grezzo è troncato; `read_tool_output` avvisa quando il tool originale aveva già troncato l'output.

4.6 La risposta era nella parte tagliata (Vivo run 2)

Problema. Nano leggeva l'articolo (circa 20.000 caratteri) ma ne vedeva circa 2.400: inizio e fine. La risposta ("Paolo Lipparelli ... 1645-1650") stava al carattere 11.647. Alla domanda di seguito ("Da parte di chi?") rispondeva "non c'è nel testo". **Causa.** Taglio "cieco" (sempre inizio + fine) e una regola nelle istruzioni ("dopo un tool il lavoro è finito") che scoraggiava ulteriori letture. **Soluzione. Taglio mirato alla domanda:** oltre a inizio e fine, il risultato più recente conserva i passaggi che contengono le parole della domanda (le parole rare pesano di più; una domanda breve usa anche quella precedente; si tiene un po' di testo prima del punto trovato per non perdere il soggetto). Eccezione esplicita alla regola: se il risultato è tagliato e la risposta non è visibile, leggere prima di dire "non c'è". **Risultato.** Verificato sul testo reale: con entrambe le domande il prompt contiene "Paolo Lipparelli" e "1645-1650". **Sul Vivo (run 3) Nano ha completato il compito.**

4.7 Modelli locali: limiti nascosti e rischio di crash

Problema. Il motore LiteRT limitava la cronologia a 3.000 caratteri qualunque fosse il modello, e tagliava solo messaggi interi. In un compito agenziale tutti i passi stanno in un unico messaggio: i risultati passavano interi e potevano superare il contesto del motore, facendo chiudere l'app (crash nativo). llama.cpp aveva lo stesso limite. **Soluzione.** Un **gestore del contesto unico** (`ContextCompactor`) usato da AI Core, LiteRT e llama.cpp, con budget proporzionato alla finestra reale del modello (massimo 32K, default 16K per Gemma 4). **Risultato sul Vivo.** Funziona, ma è troppo lento e consuma troppo (Qwen3 4B: 298 secondi di ragionamento). **Decisione: AI Core è il motore; i modelli locali restano solo come opzione.**

4.8 Aggiornare l'app senza disinstallare

Problema. Ogni build su GitHub firmava l'APK con una chiave diversa: Android rifiutava l'aggiornamento. **Soluzione.** Chiave debug fissa nel repository (privato, solo per la versione debug) e nome dell'app **Rikka-mene** per distinguerla dall'altra installazione. **Nota di sicurezza.** Chi ha accesso al repository può firmare un aggiornamento della versione debug.

4.9 Compilare senza il PC

Problema. L'ambiente cloud bloccava i repository Google e i download necessari al build. **Soluzione.** Abilitati i domini necessari nelle impostazioni dell'ambiente; Android SDK installato; mirror Google di Maven Central (Central rispondeva 429); submodule inizializzati; locale UTF-8. Workflow GitHub Actions che compila e pubblica l'APK quando un commit contiene `[release-apk]`.

5. Risultati verificati

Verifica	Dove	Risultato
Test unitari modulo ai	Gradle (cloud)	341 / 341
Test unitari app	Gradle (cloud)	1746 / 1746
Test local-llm / llama-cpp	Gradle (cloud)	128 / 128, 70 / 70
Test host + scenari agentici	JVM (cloud)	147 test + 3 scenari, tutti verdi
Compito da 45 passi senza ripetizioni	Scenario automatico	46 richieste, nessun loop
Pagina lunga, dato nel mezzo	Testo reale, offline	Dato presente nel prompt
Installazione, chat, tool, compito Mura di Lucca	Vivo X300	Superato (run 3)

Nota: gli scenari automatici usano un modello simulato che legge solo il prompt, non Gemini Nano. Dimostrano il meccanismo; il comportamento del modello reale si misura solo sul Vivo.

6. Come si usa

6.1 Installare e aggiornare

- 1 Apri la pagina **Releases** del repository su GitHub e scarica l'ultima **...-arm64-v8a-debug.apk**.
- 2 Installa sopra la versione esistente (non serve disinstallare dalla **apk-14cf4d8-run4** in poi).
- 3 Il file **.sha256** accanto serve a verificare che il download sia integro.

6.2 Configurare l'assistente

- Modello: **Gemini Nano (FULL)** del provider AICore (in alto nella chat, tocca il nome del modello).
- Tool locali dell'assistente (Impostazioni dell'assistente → Local tools): **Files, Download, Time Info, JavaScript Engine** per i test del capitolo 7. **web_fetch** è attivo di default.
- I tool che scrivono, leggono file o eseguono codice chiedono sempre l'approvazione: è voluto.

6.3 Modelli locali (opzionale, sconsigliato per l'uso quotidiano)

Impostazioni → Providers → **Local • LiteRT** → Enable → scarica **Gemma 4 E4B** (3,7 GB) → attiva Try GPU acceleration → nella chat scegli il modello locale. Ragionamento (icona lampadina) spento. Aspettati lentezza e consumo elevato.

7. Prompt di test (prossimi da fare sul Vivo)

Ogni test in una **chat nuova** su Gemini Nano (FULL). Mandare gli screenshot delle chiamate tool (tocca "Called tool ..." per vedere input e risultato).

Test A — Catena di 4 tool

Prompt:

```
Leggi con web_fetch https://it.wikipedia.org/wiki/Mura_di_Lucca e trova chi portò a termine le mura e in quali anni. Poi usa get_time_info per sapere la data di oggi, calcola con eval_javascript quanti anni sono passati dal 1650 a oggi, e salva un riassunto di una riga con write_text_file nel file /sdcard/Download/RikkaHub/test-mura.txt. Alla fine rileggi il file con read_file e dimmi cosa contiene.
```

Cosa osservare: 5 chiamate in ordine (web_fetch, get_time_info, eval_javascript, write_text_file, read_file); approvazioni richieste per eval_javascript, write_text_file e read_file; il file contiene Lipparelli e il numero di anni giusto; la risposta finale cita il contenuto riletto. Successo pieno se il file esiste e il calcolo è corretto.

Test B — Recupero da errori

Prompt:

```
Usa web_fetch per leggere https://it.wikipedia.org/wiki/Pagina_che_non_esiste_12345 e dimmi di cosa parla. Se la pagina non esiste, dimmelo e poi prova con https://it.wikipedia.org/wiki/Lucca e dimmi in una frase di cosa parla.
```

Cosa osservare: primo `web_fetch` con errore (404 o status non ok); il modello **non** ripete la stessa chiamata fallita; passa al secondo indirizzo; risposta corretta sulla città di Lucca. Fallimento se ripete la stessa URL o se dice di non poter continuare.

Test C — Protezione dai loop

Prompt:

```
Controlla con list_files cosa c'è nella cartella /sdcard/Download/RikkaHub e continua a controllare finché non compare un file chiamato fantasma.txt.
```

Cosa osservare: il file non esiste e non comparirà mai. Il sistema deve fermarsi da solo (protezione loop, limite di passi o di tempo) senza decine di chiamate identiche; idealmente il modello dice che il file non c'è dopo 1-3 controlli. Annotare quante chiamate fa e quanto tempo passa.

Test di regressione (quando cambia qualcosa)

Il test delle Mura di Lucca, seguito da "Da parte di chi?". Risposta attesa: **Paolo Lipparelli, 1645-1650.**

8. Sicurezza: cosa è protetto

- **Approvazioni:** i tool che scrivono, eseguono codice, accedono a file o alla rete chiedono conferma; le approvazioni non passano da una chat all'altra né ai sub-agent.
- **Limiti fissi** (non disattivabili dal modello): comandi distruttivi (`HardlineCommandGuard`), percorsi di sistema e dati privati dell'app (`PathSafetyGuard`), indirizzi di rete locale (protezione SSRF).
- **Risultati salvati:** leggibili solo dalla conversazione che li ha prodotti; archivio privato cancellato a ogni avvio.
- **Rischi noti non risolti** (decisioni aperte): immagini nei messaggi che caricano URL esterni automaticamente; `memory_tool` che scrive memorie persistenti senza approvazione nelle chat interattive; cartella `/tool_outputs` condivisa tra le chat con la shell.

9. Problemi aperti e prossimi passi

- 1 Test A, B, C sul Vivo (capitolo 7) e correzioni conseguenti.
- 2 Misura in logcat dei valori reali `getTokenLimit` e `counted` (richiede il Dell con adb).
- 3 Decisioni di sicurezza aperte (capitolo 8).
- 4 Valutare se unire il branch nel ramo principale (pull request).

10. Riferimenti tecnici

- **Repository:** `fmenesini/rikkahub-agent-VIVOX300`, branch `claude/fastpathrouter-storage-test-9pwqjf`.
- **Memoria di progetto** (dettagli per sviluppatori): `AGENT_PROJECT_MEMORY.md`. Build: `BUILD_ANDROID.md`.
- **File chiave:** `ai/.../providers/AICorePrompt.kt` (prompt e tagli), `AICoreProvider.kt` (`AICore`, `countTokens`), `ai/.../core/ContextCompactor.kt` (gestore contesto condiviso), `app/.../tools/ToolOutputTools.kt` (`read_tool_output` e archivio), `local-llm/.../LocalContextBudget.kt`, `.github/workflows/debug-apk-release.yml`.

- **Nuova APK:** un commit con `[release-apk]` nel messaggio avvia la build su GitHub; l'APK compare nelle Releases in circa 15 minuti.
- **Test:** `./gradlew :ai:testDebugUnitTest :app:testDebugUnitTest :app:assembleDebug`; senza SDK Android: `bash scripts/host-test/run.sh`.

Commit principali di questa fase: `3496180` locale · `36552e3` registro passi e `read_tool_output` · `eb7c9e2` `countTokens` · `301b80b/8340f75/f5f3010` pubblicazione · `fa6daa8` valori argomenti · `14cf4d8` chiave e nome · `38639e7` gestore contesto condiviso · `b9e1544` taglio mirato · `7ac252d` esito run 3.